Agent Registry, Agent Identity y Agent Gateway en Gemini Enterprise Agent Platform

Informe técnico descriptivo · Preparado el 2026-11-08 · Matías Zabaljáuregui

Documento base: la página «Govern» de Gemini Enterprise Agent Platform

Reproducción textual del original en inglés, sin traducir, de la página https://docs.cloud.google.com/gemini-enterprise-agent-platform/govern (descargada el 2026-11-08; archivo completo en `ref-gcp-docs-raw/gemini-enterprise-agent-platform__govern.md`). Se incluyen la introducción, la figura, los componentes de gobernanza y el orden de configuración recomendado. Dos cambios de formato, ninguno de contenido: la figura apunta a la copia local y los dos títulos de sección bajan un nivel. Los enlaces son los del original.

Gemini Enterprise Agent Platform provides a full suite of capabilities for the complete agentic development lifecycle, to help agent developers and enterprises Build, Scale, Govern, and Optimize their agentic applications.

Governance provides the framework for discovering, securing, and auditing AI agents and their underlying infrastructure at scale. As organizations deploy complex agentic workflows, the Govern suite of capabilities serves as the centralized command center for administrators and security teams to maintain oversight.

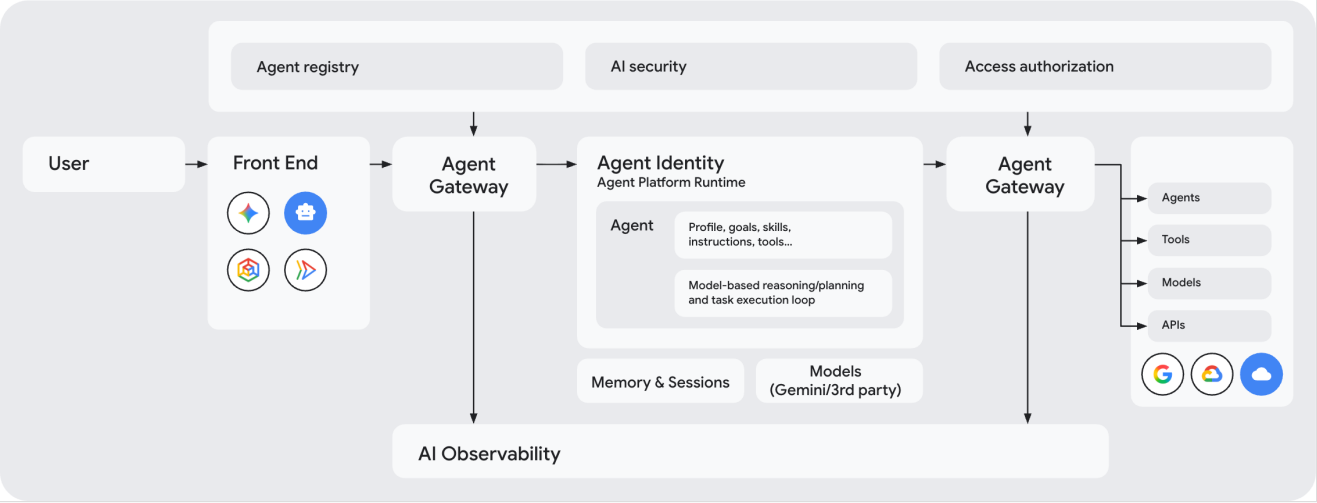


Figura 1. Agent Platform governance ecosystem. Fuente: la página citada arriba.

Core governance components

The following Agent Platform components work together to provide a unified agent governance architecture:

- **Agent Identity (Who made the request?)** : Assigns a unique, secure ID (a SPIFFE ID) to each agent. This identity acts as the agent's digital signature for authentication, access control, and auditing.

  Agent identities are secured by default with Context-Aware Access which enforces end-to-end cryptographic authentication by using mTLS and DPoP .

- **Agent Registry (What destinations are approved?)** : Acts as the central directory for all approved agents, tools, Model Context Protocol (MCP) servers, and endpoints (such as essential Google Cloud APIs) in your organization. Agent Gateway uses this directory to check permissions before allowing connections.
- **Policies (What actions are permitted?)** : Lets you implement rich sets of agentic security and governance policies that control which agents can reach specific resources and what content is allowed to pass through:
    - **IAM Unified Access Policies** : Access rules that link an agent's identity to approved tools and endpoints in Agent Registry. By default, all connections are blocked unless an explicit IAM policy grants access.
    - **Model Armor** : Content security filters attached to gateways. Model Armor scans user prompts and tool responses in real time to block prompt injection attacks, sensitive data leaks, and harmful content.
    - **Semantic governance policies** : Rules written in plain language that control how agents use tools. They enforce business rules at runtime to prevent agents from taking unintended actions, such as running unsafe combinations of tools.
    - **Token management policies** : Policies that monitor and attribute large language model (LLM) token usage, and enforce shared token budgets across your agents and applications to prevent runaway token consumption and unexpected cost spikes.
    - **Custom authorization engines** : Custom authorization extensions that let you delegate authorization decisions to third-party systems by using Service Extensions.

  Internally, all of these policy types are enforced by using authorization policies or extension bindings managed through Service Extensions.

  > Support for token management policies and extension bindings is coming soon. Features and timing are subject to change. To request early access, sign up here for early preview access .

- **Agent Gateway (Where are policies enforced?)** : Serves as the main entry and exit point for network traffic between clients-to-agents and agents-to-anywhere, and applies security and governance policies to all traffic passing through the gateway. To learn more, see How Agent Gateway enforces policies .
- **Agent observability** : Agent Gateway generates observability telemetry for all agent interactions at the network layer and exports it to Agent Observability to provide you with a comprehensive understanding of agent actions.

Recommended order of configuration

To ensure a smooth deployment process, we recommend that you follow this sequence when setting up a governed Agent Platform environment:

1. **Provision Agent Identities** : Provision workload identities (SPIFFE IDs) for your agents. For instructions, see Agent identities .
2. **Register destinations in Agent Registry** : Register target tools, MCP servers, endpoints, and agents in Agent Registry. For instructions, see Agent Registry .
3. **Configure IAM policies** : Define IAM access policies that grant agent identities (SPIFFE IDs) access to essential platform APIs and your registered destinations. For instructions, see Configure IAM access policies .

> **Tip:** For testing in audit mode, select **Audit-only** in the Google Cloud console or set `iamEnforcementMode: "DRY_RUN"` in the authorization extension `metadata` block to log evaluations without blocking access. To enable active enforcement, select **Enforce policies** in the Google Cloud console (which writes `iamEnforcementMode: "ENFORCED"` when creating a gateway, or removes `iamEnforcementMode` when updating an existing gateway), or remove `iamEnforcementMode` in your extension configuration.

4. **Configure Model Armor templates** : Set up content security templates to inspect prompt payloads and tool responses for prompt injections, jailbreaks, and sensitive data leaks. For instructions, see Configure Model Armor .

   > **Tip:** For testing in audit mode, set `enforcement_type: "INSPECT_ONLY"` in the authorization extension `metadata` block to log findings without blocking requests. Set `enforcement_type: "INSPECT_AND_BLOCK"` to start active enforcement.

5. **Configure Semantic governance policies** : Define natural language constraints that regulate tool usage and prevent unsafe tool combinations at runtime. For instructions, see Configure Semantic governance policies .

   > **Tip:** For testing in audit mode, you can set `sgpEnforcementMode: "DRY_RUN"` in the authorization extension `metadata` block to evaluate constraints in log-only mode. Remove the `sgpEnforcementMode` parameter (or set it to `"ENFORCE"`) to enforce natural language rules at runtime.

6. **Deploy Agent Gateway and monitor traffic** : Deploy Agent Gateway to begin intercepting traffic. If you enabled any of the access control mechanisms in audit mode, query Cloud Logging to generate a consolidated "would have been blocked" report and confirm that access is being evaluated as expected before transitioning your policies to active enforcement. For instructions, see Set up Agent Gateway and Audit dry-run policy violations across controls .

Nota de método y de confianza

Este informe describe tres servicios de Google Cloud que forman el núcleo de la gobernanza de agentes: **Agent Identity** (quién es el agente), **Agent Registry** (qué componentes y destinos están aprobados) y **Agent Gateway** (dónde se aplican las políticas). Está escrito a partir del texto completo de la documentación oficial y de sus diagramas.

- **Fuentes.** Páginas de `docs.cloud.google.com` descargadas el 2026-11-08 con la API de Developer Knowledge (texto completo, sin resumen intermedio) y guardadas en `ref-gcp-docs-raw/`. Las seis figuras provienen de esas páginas y están en `ref-gcp-docs-img/`. Cada figura cita su página de origen.
- **Qué se leyó a fondo.** Las páginas de visión general de los tres servicios; el modelo de datos, los conceptos, los identificadores, el ciclo de vida, el registro automático, las fronteras entre proyectos, la búsqueda, la autenticación de herramientas y la arquitectura de skills del Agent Registry; la visión general, la versión gestionada y la administración del Agent Gateway. Otras páginas (políticas IAM, SGP, delegación de autorización, Model Armor, despliegue en Runtime y Gemini Enterprise, monitoreo) se leyeron antes y están resumidas en `ref-gcp-govern-docs.md`.

- **Qué NO se hizo.** Nada de lo descrito se ejecutó en un proyecto de Google Cloud real. Lo que dice la documentación es lo que dice la documentación; no es una garantía de comportamiento.
- **Marcas usadas.**

Marca	Significado
DOC	Afirmado por la documentación citada
INFERENCIA	Deducción del autor, no dicha en la documentación
NO DOC	La documentación leída no lo cubre
VERIFICADO LOCAL	Comprobado leyendo código instalado en esta máquina (no ejecutado contra GCP)

---

1. Resumen ejecutivo

Google organiza la gobernanza de agentes en cinco piezas, cada una con una pregunta:

Pieza	Pregunta que responde	Qué es
Agent Identity	¿Quién hace la solicitud?	Identidad criptográfica por agente (SPIFFE), con certificado X.509 de 24 horas
Agent Registry	¿Qué destinos están aprobados?	Catálogo de agentes, servidores MCP, herramientas, endpoints y skills
Policies	¿Qué acciones están permitidas?	Políticas de acceso IAM (CEL), Model Armor, gobernanza semántica, gestión de tokens y motores propios
Agent Gateway	¿Dónde se aplican las políticas?	Punto de entrada y salida de red del tráfico de los agentes
Observability	¿Qué pasó?	Telemetría de red, logs y trazas

Los tres servicios de este informe se necesitan entre sí:

- La **identidad** hace que cada agente sea un *principal* que IAM puede nombrar.
- El **registro** da nombres de recurso estables a los destinos y es lo que las políticas apuntan.
- El **gateway** consulta la identidad y el registro antes de dejar pasar cada llamada, y por defecto **bloquea todo** lo que no tenga una regla explícita que lo permita.

Tres hechos que conviene tener presentes desde el principio:

1. **El estado de lanzamiento del Agent Gateway no es consistente entre páginas** (sección 6). Hasta confirmarlo en la consola, es prudente tratarlo como *Preview*.
2. **El Agent Registry no tiene un campo de responsable humano** (`owner`) ni de clasificación de datos. Guarda tipo, URL, descripción, protocolo y capacidades; no atributos de gobernanza organizacional. **DOC** (por ausencia en el modelo de datos completo).
3. **El gateway hoy solo está documentado como disponible para Agent Runtime y Gemini Enterprise.** GKE, Cloud Run y entornos multinube figuran como "próximamente". **DOC**

La Figura 1 (ecosistema de gobernanza de Agent Platform, con Agent Gateway en el centro) está al comienzo de este informe.

---

2. Agent Identity

2.1 Qué es

Agent Identity da a cada agente una **identidad criptográfica con atestación, basada en el estándar SPIFFE**. Con ella el agente se autentica ante servidores MCP, recursos de nube, endpoints y otros agentes, actuando **por cuenta propia o en nombre de un usuario final**. **DOC**

A diferencia de una cuenta de servicio, la documentación afirma que la identidad de agente:

- no se comparte entre cargas de trabajo por defecto;
- no puede suplantarse (*impersonate*);
- no permite generar claves de cuenta de servicio de larga duración;
- emite tokens de acceso de Google Cloud **ligados criptográficamente al certificado X.509 único del agente**, para impedir el robo de tokens.

Servicios compatibles: Agent Runtime, Gemini Enterprise y Cloud Run. **DOC**

2.2 Los tres contextos de identidad

La arquitectura de delegación separa tres identidades que se coordinan a lo largo de tres fronteras de confianza (Figura 2):

Identidad	Quién es	Cómo se establece
Identidad de usuario	El usuario final que inicia la acción	OAuth 2.0 u OIDC con Google Identity o un IdP de *workforce* federado
Identidad de agente	El agente actuando con autoridad propia	SPIFFE ID atestiguado y certificado X.509 aprovisionado automáticamente
Identidad delegada	El agente actuando por el usuario	Credenciales OAuth 3LO con alcance limitado, intermediadas por el *auth manager*

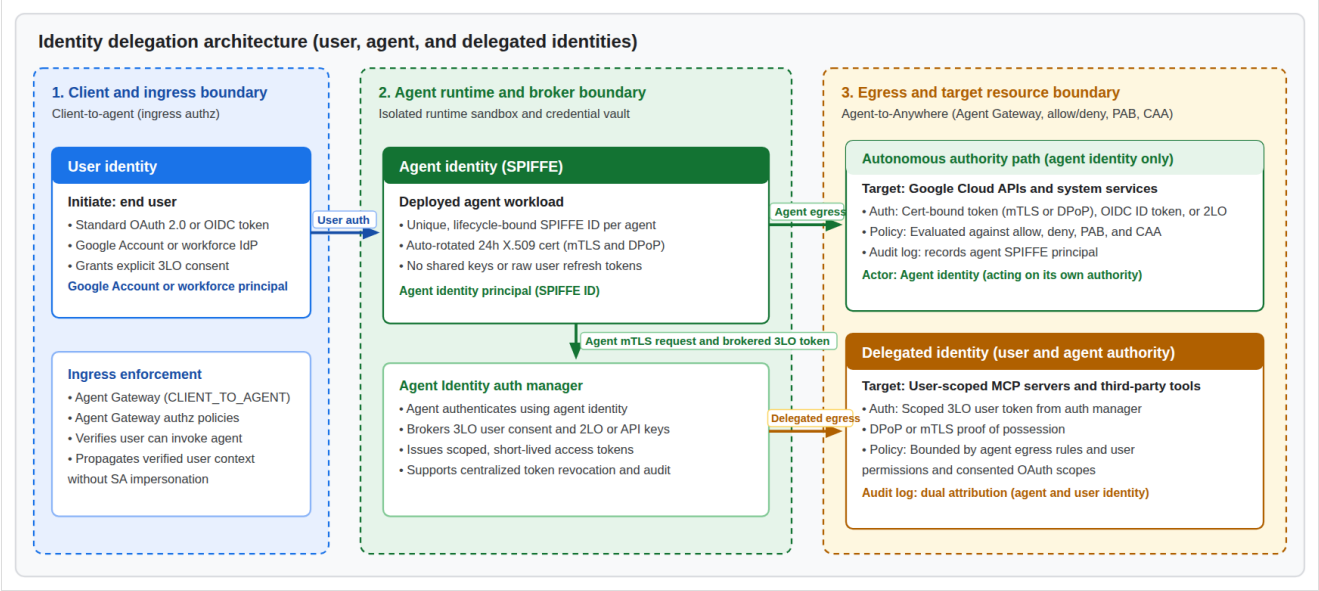


Figura 2. Arquitectura de delegación de identidad (usuario, agente y delegada). Fuente: `govern/agent-identity-overview`.

Lo que muestra la figura, en lenguaje llano:

1. **Frontera 1 (cliente e ingreso).** El usuario se autentica con OAuth/OIDC y concede un consentimiento 3LO explícito. El gateway en modo `CLIENT_TO_AGENT` verifica que el usuario pueda

invocar al agente y **propaga el contexto verificado del usuario sin suplantar una cuenta de servicio**.

2. **Frontera 2 (runtime y broker).** El agente desplegado tiene un SPIFFE ID único ligado a su ciclo de vida y un certificado X.509 que rota cada 24 horas. El *auth manager* actúa como bóveda y corredor: el agente se autentica ante él con su identidad, y él gestiona el consentimiento 3LO, las credenciales 2LO y las API keys, emite tokens de corta vida con alcance acotado y permite la revocación centralizada y la auditoría.
3. **Frontera 3 (egreso y destino).** Hay dos caminos. En el **autónomo**, el agente accede a APIs de Google Cloud con un token ligado al certificado (mTLS o DPoP), un token OIDC o 2LO; la política se evalúa contra allow, deny, PAB y CAA, y el log de auditoría registra el principal del agente. En el **delegado**, el agente accede a servidores MCP y herramientas de terceros con un token 3LO del usuario; la política queda acotada por las reglas de egreso del agente **y** por los permisos y *scopes* OAuth que el usuario concedió, y el log registra **atribución doble** (agente y usuario).

2.3 El identificador SPIFFE

```
spiffe://TRUST_DOMAIN/resources/SERVICE/RESOURCE_PATH          # identidad de la carga de trabajo
principal://TRUST_DOMAIN/resources/SERVICE/RESOURCE_PATH       # el mismo agente, escrito para IAM
```

Componente	Valor
`TRUST_DOMAIN` (con organización)	`agents.global.org-ORGANIZATION_ID.system.id.goog`
`TRUST_DOMAIN` (proyecto sin organización)	`agents.global.proj-PROJECT_NUMBER.system.id.goog`
`SERVICE`	Nombre corto del servicio: `aiplatform`, `discoveryengine`, `run`
`RESOURCE_PATH`	Ruta completa del recurso que aloja al agente

Ejemplo (Agent Runtime, con organización): `principal://agents.global.org-123456789012.system.id.goog/resources/aiplatform/projects/9876543210/locations/us-central1/reasoningEngines/my-test-agent`

Puntos que la documentación subraya:

- **El agente es el principal.** Los permisos se otorgan directamente a este identificador.
- **Al escribir un binding de IAM hay que usar la forma `principal://` o `principalSet://`, no la `spiffe://`.** La documentación del registro agrega que usar el formato SPIFFE en un binding devuelve `INVALID_ARGUMENT`. **DOC**
- **La identidad nace del recurso.** Si se elimina el agente y se despliega un reemplazo, aunque tenga el mismo nombre, código y configuración, recibe **un principal nuevo** y los permisos anteriores no le aplican. **DOC**
- **Eliminar un agente no elimina los bindings de IAM que lo mencionan:** quedan como concesiones inactivas y hay que retirarlas a mano. **DOC**

2.4 Credenciales

Credencial	Descripción
X.509	Provisionado y gestionado automáticamente; válido **24 horas**, renovado por la plataforma; encadena a una CA raíz de Google (certificados públicos en `gstatic.com`, globales y por región)
Access token de Google Cloud	Ligado criptográficamente al certificado del agente
OIDC ID token	Para autenticarse ante servicios externos por federación OIDC
Credenciales almacenadas	API keys, tokens OAuth, obtenidos del *auth manager* según el *auth provider*

Context-Aware Access (CAA). Es una política gestionada por Google, **activa por defecto**, que exige dos mecanismos:

- **mTLS** entre el agente y la API de Google Cloud (acceso directo) o entre el agente y el gateway.
- **DPoP** (*Demonstrating Proof of Possession*, RFC 9449) para el tramo que va más allá del gateway. Como mTLS termina en el gateway, el agente se reautentica con DPoP; CAA comprueba que la prueba de autorización y la prueba del recurso se firmaron con la misma clave privada del *agent identity pool*.

Con gateway habilitado, DPoP es obligatorio. **DOC** La documentación indica cómo desactivar la aplicación (variable de entorno `GOOGLE_API_PREVENT_AGENT_TOKEN_SHARING_FOR_GCP_SERVICES=False`) y la desaconseja. **DOC**

2.5 Modelos de autenticación

Autoridad	Método	Destino	Cuándo usarlo
Delegada por usuario	OAuth 2.0 de tres patas (3LO)	Herramientas externas	El agente actúa por un usuario concreto; requiere un *auth provider* 3LO
Propia del agente	Identidad en la nube (Agent Identity)	Servicios de Google Cloud	El agente accede a otros servicios de Google Cloud con su propia identidad
Propia del agente	OIDC ID token	Backends externos, APIs propias, otras nubes	Federación de identidad OIDC
Propia del agente	OAuth 2.0 de dos patas (2LO)	Herramientas externas	**Recomendado** para máquina a máquina
Propia del agente	API key	Herramientas externas	Servicios que exigen una clave
Propia del agente	HTTP basic auth	Herramientas externas	Contraseñas en texto plano: **no recomendado**

2.6 El Agent Identity auth manager

Es una **bóveda centralizada de credenciales y corredor de autenticación** para herramientas salientes. Gestiona los *auth providers*: configuraciones para adquirir, guardar y proteger API keys, client ID y secret de OAuth, y tokens OAuth delegados. El agente se autentica ante el *auth manager* con su propio SPIFFE ID, y todos los accesos de usuarios finales quedan atribuidos a ese SPIFFE ID. Automatiza el diálogo de consentimiento, da visibilidad sobre qué usuarios concedieron acceso y permite **revocarlo**. **DOC**

2.7 Controles de seguridad asociados

- **IAM** admite políticas de permiso y de denegación (*deny*).
- **Principal Access Boundary (PAB):** limita qué recursos puede alcanzar el agente, sin importar los demás permisos que tenga. Se evalúa en llamadas salientes. Un agente sujeto a PAB solo accede a lo que esa política especifique.
- **VPC Service Controls:** se pueden agregar `agentidentity.googleapis.com` y `agentidentitycredentials.googleapis.com` a un perímetro; dentro del perímetro hay que usar `restricted.googleapis.com`, y las identidades de agente sirven como principales en reglas de ingreso y egreso.
- **Auditoría:** los logs muestran la identidad del agente y, cuando actúa por un usuario, también la del usuario.

2.8 Cómo se habilita en Agent Runtime

Según la página de Agent Identity en Runtime (leída antes, resumida en `ref-gcp-govern-docs.md`):

```
agents-cli deploy --agent-identity
```

o, con el SDK, `identity_type = AGENT_IDENTITY` en la configuración del despliegue. Se verifica con:

```
gcloud ai reasoning-engines describe RESOURCE_ID --project=PROJECT_ID --location=LOCATION \
  --format="value(spec.effectiveIdentity)"
```

Sin ese indicador, la instancia sigue usando cuentas de servicio. Roles por defecto: `roles/aiplatform.agentContextEditor` y `roles/aiplatform.agentDefaultAccess`. La documentación desaconseja conceder a nivel de proyecto `roles/aiplatform.user` y `roles/aiplatform.expressUser`, porque permiten modificar o borrar otros agentes.

2.9 Límites y lagunas

- **Limitación documentada:** no se pueden otorgar a identidades de agente los roles heredados de buckets de Cloud Storage (`storage.legacyBucketReader`, etc.). **DOC**
- **NO DOC:** etapa de lanzamiento de Agent Identity (la página no la declara), cuotas y comandos para "deshabilitar" una identidad. Los mecanismos documentados para cortar el acceso son IAM *deny*, PAB, redeploy (principal nuevo) y revocación de credenciales 3LO en el *auth manager*.

---

3. Agent Registry

3.1 Qué es y para qué sirve

Agent Registry permite **registrar, descubrir y gobernar componentes agénticos** (agentes de IA, servidores y herramientas MCP, endpoints y skills independientes) en un solo registro. Su propósito declarado es eliminar el acceso fragmentado a herramientas, los datos aislados y las integraciones redundantes. Se organiza en torno a tres objetivos: **acelerar el desarrollo** (descubrir y reutilizar capacidades), **simplificar la integración** (localizar endpoints con los protocolos MCP o A2A) y **habilitar la gobernanza** (aplicar políticas de red y de acceso y límites de gestión). **DOC**

En Agent Platform, el registro mantiene la lista autoritativa de lo que hay que gobernar y **genera los identificadores y nombres de recurso** que usan los clientes de descubrimiento y las políticas de seguridad. Antes de enrutar una solicitud, **Agent Gateway consulta Agent Registry** y evalúa las políticas configuradas. **DOC**

3.2 Los cuatro tipos de componente

Componente	Qué es
Agente	Actor autónomo que planifica, razona, invoca herramientas y colabora con otros agentes
Servidor MCP	Programa que implementa el Model Context Protocol y expone herramientas y recursos de datos. Una *herramienta* es una función determinista que un agente puede invocar
Endpoint	URL de destino, normalmente una API REST, a la que accede un agente
Skill	Capacidad modular. **Independiente**: paquete de instrucciones y scripts opcionales. **A2A**: descripción de capacidad dentro de la *Agent Card* de un agente

3.3 Modelo de datos: recursos escribibles y recursos de descubrimiento

Para agentes, servidores MCP y endpoints, el registro separa la **gestión** del **descubrimiento**:

- **`Service`** (escribible): guarda los metadatos del componente: tipo, URL de la interfaz, descripción, *protocol binding* y especificación de capacidades. Se crea, actualiza o elimina para registrar o dar de baja un componente, **sin tocar el despliegue subyacente**.
- **`Agent`, `McpServer`, `Endpoint`** (de solo lectura): recursos de descubrimiento que el registro crea y actualiza a partir del `Service`. Son los que consultan orquestadores, aplicaciones y desarrolladores, y los que las políticas apuntan.

Los componentes **registrados automáticamente** no tienen `Service`: el registro ingiere los metadatos del runtime y sincroniza los cambios cuando se actualiza o elimina la carga de trabajo.

Las banderas con que se crea un `Service` deciden qué recurso resulta:

Banderas	Recurso resultante	Uso	Valores válidos
`--agent-spec-type` / `--agent-spec-content`	`Agent`	Agentes remotos A2A o REST	`a2a-agent-card`, `no-spec`
`--endpoint-spec-type` / `--endpoint-spec-content`	`Endpoint`	APIs REST o URLs externas	`no-spec`
`--mcp-server-spec-type` / `--mcp-server-spec-content`	`McpServer`	Servidores MCP externos	`tool-spec`, `no-spec`

Los pares son excluyentes. **Si se usa el par equivocado, el registro crea el recurso en la colección que coincide con la bandera**, no con la intención: pasar `--agent-spec-type=no-spec` al registrar un endpoint crea un `Agent`, y las consultas y políticas por tipo no se aplican como se espera. **DOC**

Las **skills independientes** no usan `Service`: son recursos de primer nivel `Skill`, con `SkillRevision` hijas y un `Publisher` (sección 3.9).

3.4 Cuatro capas de identificación

Es la parte más fácil de confundir, y la documentación insiste en no mezclarlas:

Capa	Propósito	Rol en la gobernanza	Sintaxis
Identificadores de registro	Descubrimiento, etiquetado y búsqueda estables	**Inmutables; no se usan para gobernar**	`urn:agent:…`, `urn:mcp:…`, `urn:endpoint:…`, `urn:skill:…`
Nombres de recurso	Identificar el destino para aplicar políticas	**Lo que exigen las políticas**, las rutas del gateway y los bindings de IAP	`projects/NÚMERO/locations/LUGAR/agents/agentregistry-ID` (y `mcpServers`, `endpoints`, `skills`)
Principales de agente	Autenticación, autorización y auditoría del llamante	Lo aplica IAM	`principal://…`, `principalSet://…`
Referencias de runtime	Enrutamiento físico y coincidencia del tráfico saliente	Se evalúan en consultas de topología y en el gateway	Atributo `agentregistry.googleapis.com/system/RuntimeReference` con `uri`

Reglas explícitas de la documentación:

- Los **URN no pueden usarse** en bindings de IAM, encabezados de autorización ni reglas de ruta del gateway.
- Un **servidor MCP no tiene principal de IAM**: solo responde solicitudes. Cuando un agente invoca una herramienta, el acceso se autoriza con el **principal del agente que llama**.
- El principal de un agente en Google Cloud **incorpora la ruta del recurso de cómputo** donde corre; por eso los permisos quedan ligados a dónde se ejecuta.

Los principales admiten **tres alcances**:

Alcance	Forma
Una instancia (un motor concreto)	`principal://agents.global.org-ORG.system.id.goog/resources/aiplatform/projects/N/locations/L/reasoningEngines/ID`
Un proyecto	`principalSet://agents.global.org-ORG.system.id.goog/attribute.platformContainer/aiplatform/projects/N`
Toda la organización	`principalSet://agents.global.org-ORG.system.id.goog/*`

Formas de URN por entorno (agentes): Agent Runtime `urn:agent:projects-N:projects:N:locations:REGIÓN:aiplatform:reasoningEngines:ID`; Cloud Run (servicios y jobs); GKE (con clúster, *namespace* y deployment); Gemini Enterprise; Google Workspace (`urn:agent:googleapis.com:locations:global:workspaceagent:workspaceagent--a2a`); y los registrados a mano (`…:agentregistry:services:ID`). **DOC**

3.5 Registro automático y manual

Característica	Registro automático	Registro manual
Alcance	Un proyecto por defecto; con *management boundary* se agrega entre proyectos miembros. Los agentes integrados de Google, los servidores MCP remotos de Google y las skills gestionadas por Google se ven en todos los proyectos	El proyecto donde se crea el recurso
Configuración	Agent Runtime, agentes integrados y servidores MCP de Google se registran solos. **GKE y Cloud Run requieren una etiqueta o bandera de opt-in**	Hay que crear un `Service` o un `Skill`
Runtimes	Agent Runtime, GKE, Cloud Run, runtimes gestionados por Google	Compute Engine, GKE, Cloud Run, on-premises, otras nubes
Componentes	Agentes, servidores MCP, skills gestionadas por Google	Agentes, servidores MCP, endpoints y skills privadas propias
Actualización de metadatos	Automática, desde el runtime o desde el endpoint de la Agent Card	**Manual**: modificar el `Service` o subir una `SkillRevision` nueva

Opt-in en GKE (por etiqueta y anotaciones en el manifiesto del Deployment):

```
metadata:
  labels:
    registry.gke.io/functional-type: "AGENT"          # el controlador de GKE registra el agente
  annotations:
    a2a-protocol.org/agent-card: |                     # descubrimiento de la Agent Card
      card: { endpoint: /.well-known/agent-card.json, protocol: HTTP, port: 8080 }
spec:
  template:
    metadata:
      annotations:
        iam.gke.io/spiffe-identity-type: agent-identity   # Agent Identity
```

Opt-in en Cloud Run: `--functional-type=agent` **junto con** `--identity-type=agent-identity`; con cualquier otro tipo de identidad, el despliegue falla. **DOC**

Qué se indexa automáticamente. Si el agente implementa **A2A**, el registro consulta `/.well-known/agent-card.json` y extrae sus habilidades A2A y capacidades. Si **no** implementa A2A (REST o gRPC), el registro lo registra pero **no puede capturar** la URL del endpoint, la identidad del agente ni sus habilidades; para asociarles capacidades hay que registrarlos a mano o registrar skills. **DOC**

Registro manual de un servidor MCP (ejemplo de la documentación; el `toolspec.json` lista `name`, `description` y `annotations` de cada herramienta, con un máximo de 10 KB):

```
gcloud agent-registry services create SERVER_NAME \
    --project=PROJECT_ID --location=REGION --display-name="DISPLAY_NAME" \
    --mcp-server-spec-type=tool-spec --mcp-server-spec-content=toolspec.json \
    --interfaces=url=SERVER_URL,protocolBinding=jsonrpc
```

Ubicación. El registro automático de servidores MCP de Google y de GKE o Cloud Run crea el recurso **solo en** **`global`**; consultarlo en una región devuelve `NOT_FOUND`. Para residencia de datos, hay que registrar a mano en la región deseada. Los gateways regionales pueden gobernar recursos regionales y servidores MCP `global`. **DOC**

3.6 Relaciones entre componentes: bindings y auth providers

- Un **binding** es una **conexión dirigida** entre un agente origen y un recurso destino (otro agente, un servidor MCP o un endpoint). Hay tres tipos: agente a agente (el orquestador invoca subagentes), agente a servidor MCP y agente a endpoint. Aportan **visibilidad sobre la topología multiagente** y ayudan a auditar qué agentes interactúan con qué.
- Un **auth provider** es una configuración del *auth manager* (API keys, OAuth) que guarda y gestiona credenciales por cuenta del agente.
- **Asociar un binding con un auth provider** permite que el agente se autentique ante destinos externos **sin incrustar secretos** en plantillas de prompt ni en el código.

```
gcloud agent-registry bindings create BINDING_NAME --project=PROJECT_ID --location=REGION \
    --display-name="DISPLAY_NAME" --source-identifier="SOURCE_ID" --target-identifier="TARGET_ID"
```

Los bindings se definen con los URN (identificadores de registro) y están acotados al proyecto donde se crean.

3.7 Descubrimiento y búsqueda

La documentación distingue dos tipos de descubrimiento: el de **consumo** (qué capacidades puede invocar un orquestador en tiempo de ejecución; es lo que hace Agent Registry) y el de **infraestructura** (dónde corren las cargas de trabajo; es lo que hace App Hub).

Modo	Qué hace
Por palabra clave	Revisa nombre visible, descripción, identificador y nombres, descripciones y etiquetas de las habilidades A2A en línea
Por prefijo	Comodín `*` al final; optimizado para baja latencia (autocompletado)
Semántica (skills independientes)	Indexa el contenido completo de `SKILL.md`; sirve a orquestadores que descubren capacidades según su intención
Por filtro	Expresiones como `displayName='…'` o `agentId='urn:agent:…'` en `gcloud agent-registry agents list --filter`

La búsqueda por palabra clave de skills mira **solo metadatos básicos** (nombre visible, descripción, nombre de recurso e identificador de skill). La búsqueda entre proyectos de un *management boundary* aparece marcada como **Preview**. **DOC**

3.8 Fronteras entre proyectos (*management boundaries*)

Por defecto, el registro opera en **un único proyecto de gobernanza**. Para gobernar muchos proyectos hay dos modelos:

- **Con *management boundary*** (Resource Manager, **Preview**): un proyecto de gestión central agrega los agentes y servidores MCP descubiertos en todos los proyectos miembros.
- **Sin *management boundary***: se designa un proyecto de gobernanza estándar y los componentes remotos se **registran a mano** allí con recursos `Service`.

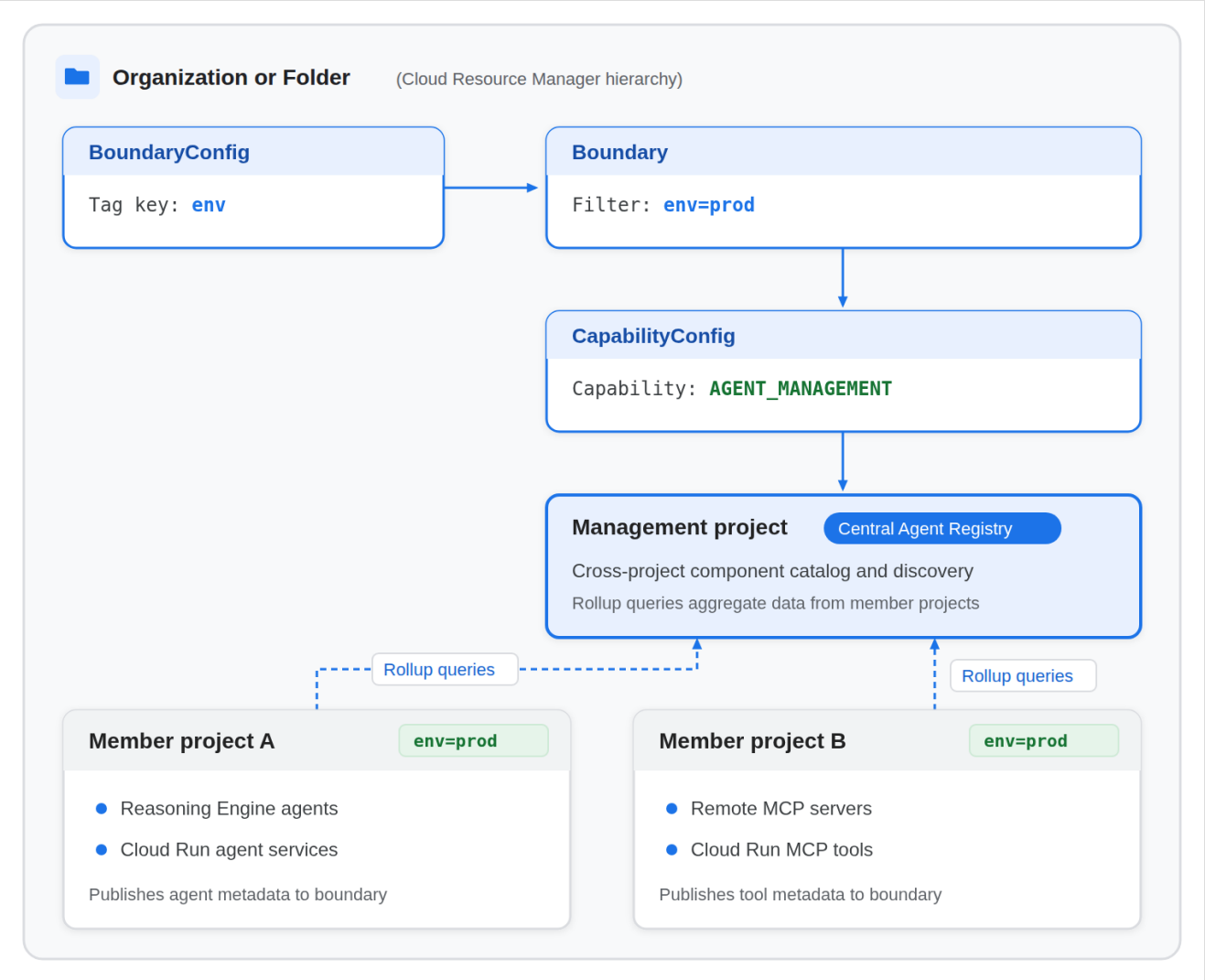


Figura 3. Frontera de gestión entre proyectos. Fuente: `agent-registry/boundaries-and-topology`.

Recursos de Resource Manager que intervienen:

Recurso	Función
`BoundaryConfig`	Establece las reglas de partición en una carpeta u organización; opcionalmente fija una clave de etiqueta (por ejemplo `env`)
`Boundary`	Perímetro de proyectos miembro; con filtro de etiqueta (`env=prod`) incluye los proyectos que tengan ese valor
`CapabilityConfig`	Liga uno o más *boundaries* a un proyecto de gestión y habilita la capacidad `AGENT_MANAGEMENT`, que activa `agentregistry.googleapis.com` en ese proyecto

Pertenencia por etiquetas. La documentación destaca que usar etiquetas da **inscripción dinámica**, **desacople de la jerarquía de carpetas**, **herencia** y **gobierno centralizado** (los permisos sobre etiquetas impiden que usuarios individuales agreguen o quiten proyectos del *boundary*). Un proyecto pertenece como máximo a un *boundary* por carpeta u organización.

Proyecto de gestión. Puede ser autogestionado (se pasa `--management-project`) o administrado por Google (Resource Manager lo crea con prefijo `google-mpf` o `google-mpo`; se elimina si se borra la `CapabilityConfig`).

Qué se agrega y qué no:

Categoría	Alcance en un *boundary*
Agentes y servidores MCP **descubiertos automáticamente**	Se agregan desde los proyectos miembro
`Service` y `Binding` **registrados a mano**	Solo en el proyecto donde se crearon (crearlos en el proyecto de gestión los hace visibles en toda la frontera)
Skills propias, `Publisher`, `Finding`, `AiApplication`	Solo en su proyecto; **no se agregan**
Skills gestionadas por Google	Visibles en todos los proyectos

El *rollup* no cruza ubicaciones: una consulta en `us-central1` no agrega recursos de `global` ni de `us`.

Control de acceso. Se otorgan los roles de descubrimiento en el proyecto de gestión, sin requerir acceso a cada proyecto miembro. Como los `Service` y `Binding` creados en un proyecto miembro no suben al de gestión, la documentación indica que se puede **restringir su creación en proyectos miembro con una restricción de Organization Policy o una política de denegación de IAM**. Las restricciones de VPC Service Controls se aplican solo a las interacciones con el registro en el proyecto de gestión.

3.9 Skills independientes y su gobernanza

Las skills siguen la **especificación abierta Agent Skills**: un archivo ZIP con un `SKILL.md` en la raíz (metadatos YAML e instrucciones en lenguaje natural) y scripts opcionales.

Elemento	Qué es
`Publisher`	Establece procedencia, nivel de verificación y prefijo de nombre. **`FIRST_PARTY`**: publicadores verificados por Google (`cloud.google.com`, `discoveryengine.googleapis.com`). **`PRIVATE`**: publicador por defecto de las skills propias, con prefijo `private-`. No se pueden crear publicadores propios
`Skill`	Contenedor estable: nombre, descripción, `skillId`, publicador, **`defaultRevision`**, **`targetState`** y **`scanningConfig.policy`**
`SkillRevision`	Instantánea **inmutable** del paquete. Subir un ZIP nuevo crea una revisión; los agentes fijados a una revisión quedan protegidos de cambios inesperados; se puede revertir `defaultRevision`
`Finding`	Hallazgo de seguridad de solo lectura, con clase `VULNERABILITY` o `SECRET`, severidad, archivos y líneas

Estados de ciclo de vida: `DRAFT` (no se sirve), `ACTIVE` (se sirve y se indexa), `DISABLED` (no se sirve; reversible), `DEPRECATED` (sigue disponible; se desmantelará tras un período de gracia), `DECOMMISSIONED` (**terminal, irreversible**; solo desde `DEPRECATED`).

Políticas de escaneo (se configuran por skill):

Política	Comportamiento
`ENABLED_BLOCKING` (**por defecto**)	Escanea cada revisión (exfiltración de datos, credenciales expuestas, inyección de prompts); si hay hallazgos, **bloquea la descarga y la promoción a revisión por defecto**. Una skill creada como `ACTIVE` arranca en `DRAFT` hasta que termina el escaneo sin hallazgos
`ENABLED_NONBLOCKING`	Modo de auditoría: publica hallazgos pero permite descargas y promociones con avisos
`DISABLED`	Sin escaneo

Controles de organización. Se puede exigir **CMEK** para los payloads de skills (ubicaciones `us` y `eu`; **Preview**; debe configurarse antes de crear la primera skill de esa ubicación) mediante `constraints/gcp.restrictNonCmekServices` y `constraints/gcp.restrictCmekCryptoKeyProjects`. Para impedir skills sin

escaneo, **no hay restricción gestionada**: hay que definir una **restricción personalizada de Organization Policy sobre el recurso `Skill`** (por ejemplo, denegar `resource.scanningConfig.policy == "DISABLED"`, con excepciones por etiquetas). Las políticas de organización **no pueden inspeccionar revisiones, contenidos ni hallazgos**. **DOC**

Skills independientes vs. habilidades A2A:

	Skill independiente	**Habilidad A2A en línea**
Recurso	`Skill` de primer nivel con `SkillRevision`	Arreglo anidado dentro del recurso `Agent`
Contenido	ZIP con `SKILL.md`, scripts y activos	Objeto JSON en la Agent Card (`id`, `name`, `description`)
Versionado	Revisiones inmutables y puntero `defaultRevision`	Atado a las actualizaciones de `agent-card.json`
Ejecución	Se descarga y carga en el contexto o runtime del agente	Anuncia capacidades para delegación entre agentes
Gobierno	Independiente del despliegue de cualquier agente	Acoplado al ciclo de vida del agente anfitrión

3.10 Roles de IAM

Rol	**Alcance**
`roles/agentregistry.viewer`	Ver y buscar agentes, servidores MCP, herramientas, skills, publicadores, endpoints y bindings
`roles/agentregistry.editor`	Ver y editar: registrar, actualizar y eliminar componentes y revisiones de skills
`roles/agentregistry.user`	Ver todo y gestionar skills, revisiones y configuraciones de escaneo, **sin** editar agentes, servidores MCP ni endpoints
`roles/agentregistry.admin`	Todas las acciones, incluidos hallazgos de seguridad, cifrado de skills, bindings y políticas IAM de skills

Para el acceso de **egreso**, el rol `roles/iap.egressor` (otorgado sobre el recurso registrado) es el que habilita a un agente a alcanzar un destino a través del gateway. **DOC**

3.11 Cómo lo consume ADK

El código de ADK instalado en esta máquina (versión 2.14) incluye el módulo `google.adk.integrations.agent_registry` con la clase `AgentRegistry`. **VERIFICADO LOCAL** (leído en el fuente, no ejecutado contra un registro real):

- Métodos de consulta: `list_mcp_servers`, `search_mcp_servers`, `get_mcp_server`, `list_endpoints`, `get_endpoint`, `list_agents`, `search_agents`, `get_agent_info`.
- Métodos de uso: `get_mcp_toolset(...)` (devuelve un `McpToolset` con las herramientas del servidor registrado), `get_model_name(endpoint_name)` y `get_remote_a2a_agent(...)`.
- Al importar el módulo en esta máquina aparece el error "*Missing required dependencies for Agent Identity Auth Manager. Please install with: pip install "google-adk[agent-identity]"*": **requiere el extra `agent-identity`**, que no está instalado.

Uso según la documentación (el ADK resuelve los bindings configurados en el registro, incluido el *auth provider*):

```
from google.adk.integrations.agent_registry import AgentRegistry
registry = AgentRegistry(project_id=project_id, location=location)
my_mcp_toolset = registry.get_mcp_toolset(mcp_server_name="mcpServers/SERVER_ID")
```

3.12 Límites y lagunas

- **Estado de lanzamiento:** la página de visión general no trae etiqueta; **sí** están marcados *Preview* el *management boundary*, el modelo de datos entre proyectos, la búsqueda entre fronteras y CMEK para skills. Las skills independientes figuran como *Preview* en la página `govern/agent-registry`. **DOC**
- **NO DOC (modelo de datos):** `owner`, etiquetas de usuario, clasificación de datos, nivel de autonomía o centro de costos. Los metadatos de un `Service` son los de la sección 3.3.
- Los recursos registrados automáticamente son de solo lectura; para cambiarlos hay que cambiar la carga de trabajo.
- Mover el registro a otro proyecto no migra datos: hay que recrear los recursos.

---

4. Agent Gateway

4.1 Qué es

Agent Gateway es la **capa de gobernanza y aplicación de seguridad** para agentes: el **punto de entrada y salida de red definido por software** para todas las interacciones agénticas, ya sea entre usuarios humanos y agentes, entre agentes y herramientas MCP, o entre agentes autónomos. Permite aplicar una postura de **mínimo privilegio** consistente. **DOC**

Beneficios declarados:

- **Para desarrolladores:** no gestionar enrutamiento VPC, configuración de Envoy ni rotación de certificados mTLS; mediación entre MCP, A2A, REST y gRPC; compatibilidad con cualquier framework; terminación automática de mTLS e integración con el *auth manager* (OAuth 2.0 y Workload Identity Federation).
- **Para seguridad y plataforma:** gobernanza centralizada entre runtimes, protecciones de IA (Model Armor), protección de perímetro y contra exfiltración (adjuntos de red VPC privada y reglas de VPC Service Controls), y observabilidad de extremo a extremo con trazas OpenTelemetry de capas 7 y 8 en Cloud Logging y Cloud Trace.

4.2 Dos rutas de acceso gobernadas

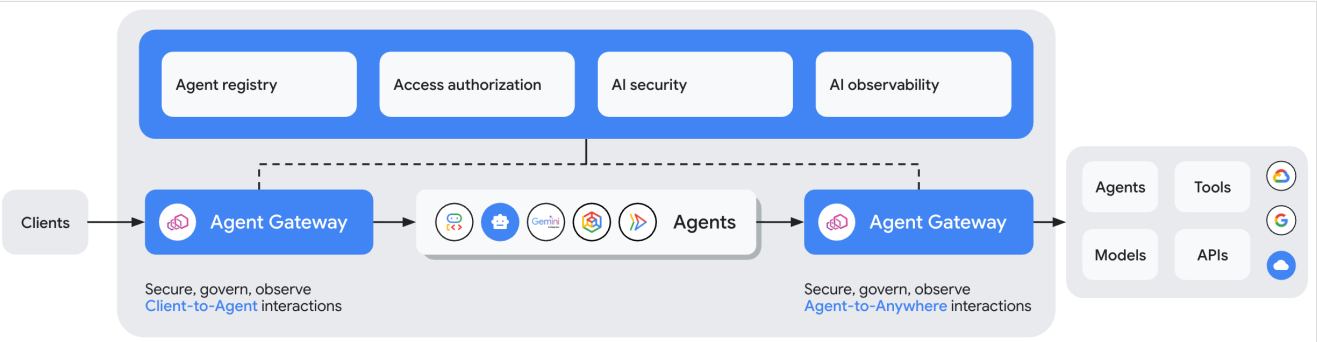


Figura 4. Modos de operación de Agent Gateway. Fuente: `govern/gateways/agent-gateway-overview`.

Ruta	Qué protege	Qué aplica
Client-to-Agent (ingreso)	Comunicación entre clientes (web, CLI, sistemas externos) y agentes o herramientas MCP alojados	Actúa como punto frontal: autentica credenciales entrantes e inspecciona prompts
Agent-to-Anywhere (egreso)	Solicitudes salientes de los agentes a herramientas externas, servidores MCP remotos, APIs SaaS u otros agentes	Identidad ligada a la carga de trabajo, cumplimiento del registro de destinos, políticas IAM y sanitización del contenido

Cada modo es **independiente**; una instancia de Agent Runtime puede estar vinculada a un gateway de egreso y a uno de ingreso a la vez.

Qué hace cada capa de gobernanza en cada ruta (documentación del gateway gestionado):

Capa	Client-to-Agent	Agent-to-Anywhere
Identidad	Identidad o credenciales del cliente que pasa al gateway	*Agent identity* ligada a la carga de trabajo (SPIFFE o token WIF)
Agent Registry	**No aplica**	Registra los destinos salientes (herramientas, servidores MCP, agentes)
Políticas de acceso IAM	Se evalúan según la identidad del cliente y el agente destino	Las evalúan IAP y las políticas PAB según la identidad del agente y el destino
Model Armor	Escanea prompts entrantes (inyección, jailbreak, contenido dañino)	Inspecciona payloads de herramientas y respuestas del agente
Gobernanza semántica	Evalúa restricciones en lenguaje natural contra la solicitud del cliente	Las evalúa contra acciones de herramientas y delegaciones salientes

4.3 Modelos de despliegue

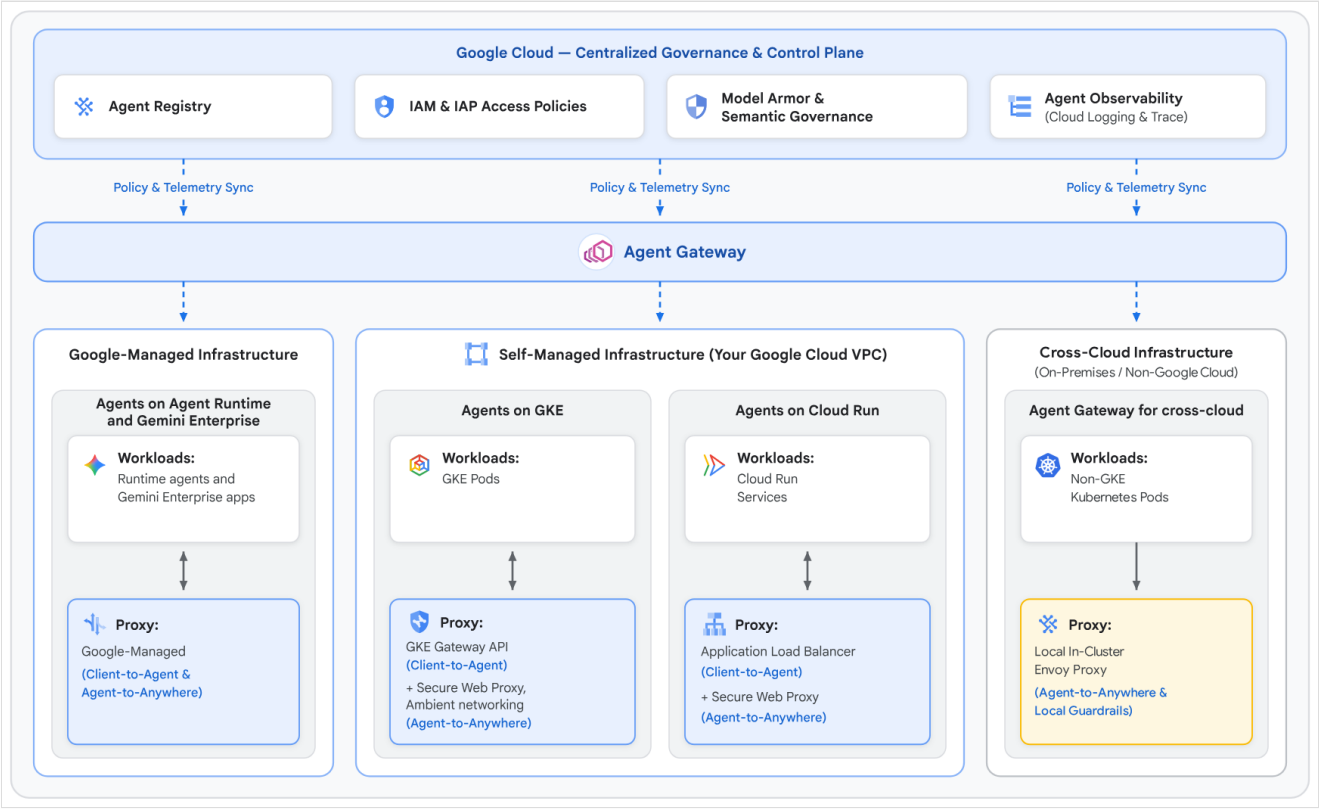


Figura 5. Modelos de despliegue entre fronteras de infraestructura. Fuente: `govern/gateways/agent-gateway-overview`.

Entorno del agente	Modelo	Infraestructura de proxy	Estado en la página
Agent Runtime y Gemini Enterprise	Agent Gateway on Google Cloud, **gestionado por Google**	No aplica	"Disponible actualmente"

Entorno del agente	Modelo	Infraestructura de proxy	Estado en la página
GKE	On Google Cloud, **autogestionado**	Ingreso: GKE Gateway API (balanceador regional). Egreso: Secure Web Proxy (entre clústeres y externo) y *ambient networking* sin sidecars (tráfico pod a pod)	"Próximamente"
Cloud Run	On Google Cloud, **autogestionado**	Ingreso: Application Load Balancer. Egreso: Secure Web Proxy; el tráfico se intercepta con *ambient networking* y viaja por Direct VPC egress dentro de un túnel mTLS que lleva la identidad del agente	"Próximamente"
Multinube (EKS, AKS, OpenShift, centros de datos)	**Agent Gateway for cross-cloud**	Proxy Envoy autogestionado en el clúster del cliente	"Próximamente"

En el modelo multinube, **el plano de gestión está en Google Cloud** (identidades, registro, políticas), pero el gateway corre en el clúster del cliente; el tráfico del agente **no sale de su entorno** y solo las verificaciones configuradas (por ejemplo, Model Armor) llaman a Google Cloud. Intercambia credenciales de carga de trabajo de Kubernetes por credenciales de Agent Identity mediante Workload Identity Federation. **DOC**

Protocolos. MCP (SSE o HTTP POST), A2A y REST/gRPC (HTTP/1.1 y HTTP/2). Para MCP, el gateway **inspecciona el JSON-RPC** para extraer el nombre de la herramienta y el método, lo que permite políticas finas por herramienta. **DOC**

4.4 Cómo aplica las políticas

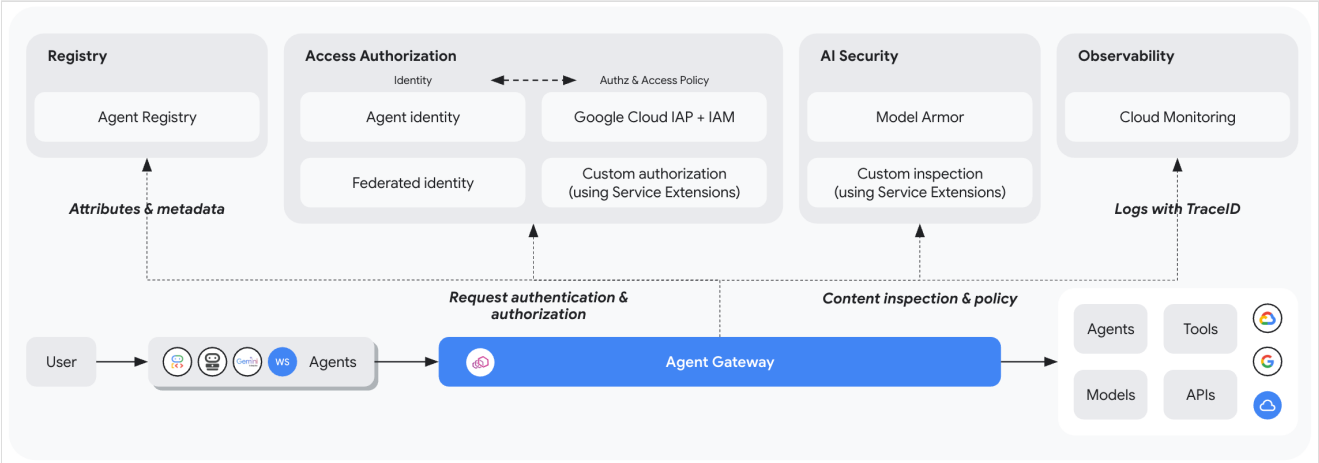


Figura 6. Control de acceso con Agent Gateway. Fuente: `govern/gateways/agent-gateway-managed-overview`.

Secuencia en egreso (Agent-to-Anywhere) según la documentación:

1. **Solicitud saliente e interceptación.** El agente, identificado por su *agent identity*, hace una llamada; el gateway en modo egreso la intercepta.
2. **Verificación de IAM, PAB e IAP.** IAP comprueba que exista una política de acceso IAM que conceda a la identidad del agente el permiso `iap.resources.egressViaIAP` sobre el destino. También se evalúan las políticas de **Principal Access Boundary**: un agente sujeto a PAB solo puede acceder a los recursos que esas políticas especifican.
3. **Verificación del registro.** El gateway comprueba que el destino esté registrado en Agent Registry (o resuelve la URL si no lo está). Registrar los destinos es lo recomendado para aplicar políticas por recurso y controles por herramienta. **Si el destino no está registrado, debe**

existir una política IAM explícita sobre la URL. Por defecto, si no hay política que coincida, la solicitud se deniega.

4. **Inspección del contenido.** Si lo anterior pasa, se evalúan las barreras opcionales: **Model Armor** (payloads de herramientas y respuestas), **políticas de gobernanza semántica** (restricciones en lenguaje natural contra invocaciones de herramientas y acciones del agente) y **motores de autorización propios** mediante Service Extensions (uno o más pueden permitir o denegar).
5. **Reenvío.** Si todo pasa, el gateway reenvía la solicitud al destino.

Secuencia en ingreso (Client-to-Agent), solo para agentes en Agent Runtime: el cliente envía la solicitud, el gateway la intercepta como *frontend* del agente, evalúa la política de autorización adjunta (**Model Armor** escanea el prompt; la **gobernanza semántica**, si se configura *en lugar de* Model Armor, evalúa las restricciones) y, si pasa, reenvía al agente.

Internamente, todas las políticas se aplican como *authorization policies* o *extension bindings* gestionadas por Service Extensions. Dos perfiles: `REQUEST_AUTHZ` (identidad y control de acceso, con acceso a las cabeceras) y `CONTENT_AUTHZ` (inspección profunda de contenido: Model Armor, gobernanza semántica o un motor propio). En egreso se evalúa primero `REQUEST_AUTHZ` y luego `CONTENT_AUTHZ`. **DOC**

4.5 Las políticas disponibles

Política	Qué hace	Estado según la documentación
IAM Unified Access Policies	Reglas ALLOW/DENY que ligan la identidad del agente con herramientas y endpoints registrados; condiciones en CEL sobre nombre de herramienta, método MCP, anotaciones, recurso registrado y destino no registrado. **El DENY prevalece**	Descrita sin etiqueta de etapa
Model Armor	Plantillas de filtros sobre prompts y respuestas; modos `INSPECT_ONLY` e `INSPECT_AND_BLOCK`	Descrita sin etiqueta de etapa
Gobernanza semántica (SGP)	Restricciones en lenguaje natural evaluadas por un LLM sobre las llamadas a herramientas que el modelo propone	**Preview**
Gestión de tokens	Monitoreo, atribución y presupuestos compartidos de tokens de LLM	**Próximamente** (acceso anticipado)
Motores de autorización propios	Delegación de decisiones a terceros mediante Service Extensions (`ext_proc`)	Las *extension bindings* figuran como "próximamente"; las extensiones por política `CUSTOM` se muestran con comandos `gcloud`

Detalles de las políticas de acceso IAM y de SGP (atributos CEL, comandos y límites) están en `ref-gcp-govern-docs.md`, secciones 3.1 y 3.2.

4.6 Puesta en marcha mínima (egreso gestionado)

Gateway:

```
name: AGENT_GATEWAY_NAME
protocols: [MCP]
googleManaged:
  governedAccessPath: AGENT_TO_ANYWHERE      # o CLIENT_TO_AGENT
registries:
  - //agentregistry.googleapis.com/projects/PROJECT_ID/locations/LOCATION
```

```
gcloud network-services agent-gateways import AGENT_GATEWAY_NAME --source=gw.yaml --
location=LOCATION
```

Autorización con IAP (extensión y política `CUSTOM` con perfil `REQUEST_AUTHZ`, con `failOpen: false` y `timeout: 1s`; para auditar sin bloquear se agrega `iamEnforcementMode: "DRY_RUN"` en la metadata de la extensión).

Recomendación de la documentación: empezar en modo auditoría (*Audit-only* / `DRY_RUN`), revisar los logs y pasar a *Enforce policies* después. **Advertencia importante:** al vincular un gateway en modo de aplicación, **todo el tráfico saliente del agente pasa por él, incluida la API de Sessions**; sin una lista de los endpoints esenciales permitidos, las invocaciones fallan con error 498. **DOC**

4.7 Administración posterior al despliegue

Hay configuraciones **inmutables** que obligan a desplegar un gateway de reemplazo:

Configuración	Mutabilidad
Nombre y región del gateway	Inmutables
Ruta de acceso gobernada (`AGENT_TO_ANYWHERE` o `CLIENT_TO_AGENT`)	**Inmutable**
Adjunto de red de Private Service Connect	Inmutable una vez asociado
Plantilla de conectividad (DNS peering, confianza TLS, egreso VPC)	Condicionalmente modificable, mientras use el **mismo** adjunto de red
Registros asociados, descripción y etiquetas	Modificables (se exporta el YAML, se edita y se reimporta)

Dentro de un mismo proyecto y región, **todos los agentes de Agent Runtime deben vincularse al mismo gateway**; no se admite pasar de un gateway a otro en el lugar (hay que desvincular todos, esperar y revincular). **DOC**

4.8 Observabilidad

- **Logs:** `networkservices.googleapis.com/gateway_requests` (decisiones de permitir y denegar), `cloudaudit.googleapis.com/data_access` (decisiones `AuthorizeUser` de IAP) y los logs de stdout y stderr del motor de razonamiento. Los logs de Model Armor se emiten por separado.
- **Campos útiles** en `jsonPayload`: `agentGatewayInfo`, `mcpInfo` (método MCP y nombre de herramienta), `agentRegistryResource` y `serviceExtensionsInfo`.
- **Métrica:** `networkservices.googleapis.com/agentgateway/total_latencies`.
- **Panel:** pestaña *Observability* del gateway, con intentos y fallos de autorización, solicitudes por segundo y denegaciones 403.
- **Trazas:** mediante una política de telemetría (`telemetryPolicies`) hacia Cloud Trace, correlacionable con el campo `trace` de cada log.

4.9 Límites y restricciones documentadas

- Cada instancia puede gobernar hasta **5.000 recursos** registrados en Agent Registry.
- Egreso: máximo **4 políticas de autorización** por gateway. Ingreso: máximo **una** política `CONTENT_AUTHZ`.
- En Gemini Enterprise **no se admite** el modo Client-to-Agent.
- No se puede vincular un gateway a motores de razonamiento creados **antes del 2026-04-29**.
- Cambiar la vinculación **archiva de forma irreversible** las revisiones existentes.

- Security Command Center *Agent Engine Threat Detection* no está disponible con el gateway habilitado.
- Gateways creados después del 2026-09-08 con plantilla de conectividad requieren **egreso por VPC** para usar VPC Service Controls.
- La gobernanza semántica **no admite respuestas en streaming**.

---

5. Cómo se integran: el recorrido de una llamada

Un agente en Agent Runtime llama a una herramienta de un servidor MCP registrado. Qué pasa y quién interviene, según la documentación:

Paso	Qué ocurre	Pieza
0. Antes	El agente se desplegó con `identity_type=AGENT_IDENTITY` y `agent_gateway_config`; se registraron los destinos; hay una política IAM que concede `egressViaIAP`	Identity, Registry, Gateway
1	El agente llama; su tráfico sale hacia el gateway con su certificado X.509 (mTLS)	Identity
2	El gateway verifica la identidad y evalúa las políticas IAM y PAB: ¿esta identidad puede llegar a este destino?	Gateway + IAP
3	Consulta el registro: ¿el destino está aprobado? Extrae del JSON-RPC el método y la herramienta	Gateway + Registry
4	Se evalúan reglas ALLOW/DENY por herramienta (el DENY gana); sin regla, se deniega	Políticas de acceso
5	Inspección de contenido: Model Armor y, si está activa, gobernanza semántica o un motor propio	Políticas de contenido
6	El gateway reenvía con DPoP (el tramo posterior al gateway)	Identity (CAA)
7	Todo queda en `gateway_requests`, en los logs de auditoría y en la traza	Observabilidad

Qué pregunta responde quién (recordatorio para el diseño):

Pregunta de gobernanza	Respuesta en la plataforma
¿Quién es este agente?	Agent Identity (SPIFFE, X.509 de 24 h)
¿Qué componentes existen?	Agent Registry (descubrimiento automático y manual)
¿Qué puede alcanzar este agente?	Políticas de acceso IAM sobre recursos registrados + PAB + *default deny* del gateway
¿Qué está pasando ahora mismo?	Logs del gateway, auditoría de IAP, trazas y panel
¿Qué pasó con el contenido?	Model Armor (logs `sanitize_operations`) y SGP (`semantic-governance-policy`)

---

6. Estado de lanzamiento y contradicciones de la documentación

Tema	Lo que dicen las páginas	Cómo tratarlo
Estado de Agent Gateway	La página de visión general: "disponible actualmente para agentes que corren en Gemini Enterprise Agent Platform", sin etiqueta. La plantilla de App Design Center: **Private Preview**, con formulario de acceso. La página de Context-Aware Access: **Preview**	**POR CONFIRMAR** en la consola. Hasta entonces, "Preview"
Agent Registry	Sin etiqueta en la visión general; *Preview* en *management boundary*, modelo entre proyectos, búsqueda entre fronteras y CMEK de skills	Base utilizable; funciones entre proyectos en Preview
Agent Identity	Sin etapa declarada	NO DOC
Atributo CEL de "solo lectura"	La referencia de políticas usa `destination.agent_registry.mcp_server.tool.annotations.read_only_hint`; el codelab menciona `iap.googleapis.com/mcp.tool.isReadOnly`	Usar el de la referencia hasta probar en un proyecto
Nombre de recurso de un agente	El registro define `agents/agentregistry-ID`; un ejemplo de política usa `agents/customer-support-agent`	INFERENCIA: el ejemplo es ilustrativo; copiar el nombre real desde `gcloud agent-registry agents list`
SGP e interceptación	Dice que el gateway intercepta agente–modelo y agente–herramientas, pero la política de ejemplo solo envía al motor las rutas de inferencia del modelo	Lo que SGP ve de las llamadas a herramientas es lo que el **modelo propone** en su respuesta

7. Lo que la documentación no cubre

- **Metadatos de gobernanza en el registro:** `owner`, clasificación de datos, autonomía, centro de costos.
- **Políticas sobre argumentos, montos y clasificación** de datos de forma determinista (las condiciones CEL no los ven).
- **Estados del plano de control** (suspender, poner en cuarentena, drenar) como acciones nativas sobre un agente.
- **Un mecanismo documentado para "deshabilitar" una identidad.** Existen IAM *deny*, PAB, redeploy y revocación 3LO.
- **Cómo llega la identidad del agente a un motor de autorización propio**, el esquema de mensajes y la semántica de `failOpen`.

- **Cuotas distintas de las 5.000 entradas** por gateway, latencia por evaluación y regiones disponibles del gateway.

8. Glosario

Término	Significado
Agente	Actor autónomo que planifica, razona e invoca herramientas
Agent Card	Documento A2A (`/.well-known/agent-card.json`) con el que un agente declara capacidades e identidad
Auth manager / auth provider	Bóveda de credenciales de Agent Identity; un *auth provider* es una configuración para obtener y guardar una credencial concreta
Binding (del registro)	Conexión dirigida entre un agente origen y un recurso destino, con *auth provider* opcional
CAA	Context-Aware Access: exige mTLS y DPoP
CEL	Common Expression Language; lenguaje de las condiciones de las políticas de acceso
DPoP	Prueba de posesión de la clave (RFC 9449), ligada al token
Frontera de gestión	`Boundary` de Resource Manager que agrupa proyectos para la gobernanza central del registro
IAP	Identity-Aware Proxy; evalúa las políticas de acceso en el gateway
Identificador de registro	URN inmutable para descubrimiento; no sirve para políticas
MCP	Model Context Protocol
Nombre de recurso	Identificador estándar de Google Cloud que usan las políticas
PAB	Principal Access Boundary: límite de recursos alcanzables por una identidad
Principal de agente	Identidad del agente en IAM (`principal://…`)
Proyecto de gobernanza / de gestión	Proyecto donde se activa y consulta el registro; el de gestión es el central de una frontera
Referencia de runtime	Ruta del cómputo donde corre un componente
SGP	Semantic Governance Policy: restricciones en lenguaje natural
SPIFFE	Estándar de identidad de cargas de trabajo

9. Fuentes

Documentación (texto completo en `ref-gcp-docs-raw/`):

- Agent Registry: https://docs.cloud.google.com/agent-registry/overview · `/data-model` · `/concepts` · `/identity-and-identifiers` · `/component-lifecycle-and-discovery` · `/boundaries-and-topology` · `/skills-architecture` · `/automatic-registration` · `/manual-registration` · `/search-agents-and-tools` · `/authenticate-toolsets`
- Agent Gateway: https://docs.cloud.google.com/gemini-enterprise-agent-platform/govern/gateways/agent-gateway-overview · `/agent-gateway-managed-overview` · `/manage-agent-gateways`
- Agent Identity: https://docs.cloud.google.com/gemini-enterprise-agent-platform/govern/agent-identity-overview
- Complementos leídos antes (resumen en `ref-gcp-govern-docs.md`): políticas IAM (UAP), gobernanza semántica, delegación de autorización, Model Armor, despliegue en Runtime y en Gemini Enterprise, monitoreo, Context-Aware Access y plantilla de Application Design Center.

Figuras: las seis provienen de las páginas citadas bajo cada una (`docs.cloud.google.com/static/…`); las tres en formato SVG se convirtieron a PNG con Chrome para incluirlas en este documento.

Verificación local: lectura del fuente de `google-adk` 2.14.0 en `~/.local/lib/python3.14/site-packages/google/adk/integrations/agent_registry/agent_registry.py`.

---

10. Anexo: qué significa el extra `agent-identity` y tres ejemplos de uso de `AgentRegistry` en ADK

10.1 Qué significa «exige el extra `google-adk[agent-identity]`»

El código para hablar con Agent Registry viene incluido en ADK 2.14 (`google.adk.integrations.agent_registry`), pero necesita librerías adicionales que **no están instaladas** en esta máquina.

Un *extra* es un grupo de dependencias opcionales de un paquete de Python. `google-adk` instala lo básico; las funciones que requieren librerías más pesadas o especializadas se activan pidiendo el extra al instalar:

```
pip install "google-adk[agent-identity]"
```

Al importar el módulo en esta máquina, ADK lanzó: "*Missing required dependencies for Agent Identity Auth Manager. Please install with: pip install "google-adk[agent-identity]*"". El módulo usa el *auth manager* de Agent Identity para obtener las credenciales de las herramientas, y esa parte necesita las librerías del extra.

- **Sí está comprobado:** la clase existe y se leyeron sus métodos en el fuente. **VERIFICADO LOCAL**
- **No está comprobado:** su ejecución. Exige instalar el extra y contar con un proyecto de Google Cloud con Agent Registry habilitado.
- **Para el curso:** si los alumnos usan `AgentRegistry`, el `pip install` de la clase debe incluir `google-adk[agent-identity]`.
- **Precaución:** ya hubo un conflicto de versiones entre `google-cloud-aiplatform` y `google-adk`; instalar el extra puede mover más versiones. Conviene hacerlo en un entorno virtual aparte, con una simulación previa (`pip install --dry-run`).

10.2 Tres ejemplos cortos

Estos ejemplos usan las firmas leídas en el fuente de ADK 2.14. **No se ejecutaron**, porque requieren el extra instalado y un proyecto con Agent Registry.

Ejemplo 1. Darle a un agente las herramientas de un servidor MCP registrado, sin cablear URLs ni credenciales

```
from google.adk.agents import Agent
from google.adk.integrations.agent_registry import AgentRegistry

registry = AgentRegistry(project_id="MI_PROYECTO", location="us-central1")
tools = registry.get_mcp_toolset(mcp_server_name="mcpServers/SERVER_ID")
agent = Agent(name="analista", model="gemini-2.5-flash", tools=[tools])
```

El registro resuelve la URL del servidor y los bindings. Si hay un *auth provider* asociado, ADK maneja la autenticación. Es lo que muestra la documentación.

Ejemplo 2. Descubrir un agente por intención y delegarle una tarea

```
res = registry.search_agents(search_string="resolver reclamos de seguros",
                             search_type="SEMANTIC")
# elegir un resultado y tomar su nombre de recurso (agents/ID)
especialista = registry.get_remote_a2a_agent(agent_name="agents/ID")
```

Un orquestador encuentra subagentes sin una lista fija en el código. `search_type` acepta `KEYWORD` o `SEMANTIC`. La documentación dice que la búsqueda semántica indexa el contenido completo en el caso de las **skills**; para agentes no lo confirma.

Ejemplo 3. Auditar qué hay registrado, por ejemplo para encontrar servidores MCP sin revisar

```
for s in registry.list_mcp_servers(filter_str="displayName:finance*").get("mcpServers", []):
    print(s["name"], s.get("displayName"))
```

Sirve para el inventario de la clase 1, en la sección del *shadow agent sprawl*. El nombre de la clave de la respuesta (`mcpServers`) y la sintaxis exacta del filtro se tomaron de la convención de la API y de ejemplos de la documentación: **son lo primero a confirmar al probarlo**.

Límite común a los tres: ninguno impone una política. El control de qué puede alcanzar cada agente sigue en el gateway y en las políticas de IAM.